

ZOMBIE SURVIVAL

Juego de supervivencia con programación concurrente en Python

Integrantes:

OCAMPO, Nicole Fabiana - 44451238 - nocampo@alumno.unlam.edu.ar
LOIOTILE, Juan Cruz - 42101782 - jloiotile@alumno.unlam.edu.ar
DA SILVA, Lautaro - 42816815 - ldasilva@alumno.unlam.edu.ar
MARTUCCI, Federico - 44690247 - fmartucci@alumno.unlam.edu.ar

Asignatura:

(3634) PROGRAMACIÓN CONCURRENTE — 1° Cuatrimestre 2026

Grupo: M1

Repositorio GitHub:

<https://github.com/TeoFrancisTurri/survival-game>

Fecha - Versión:

05/07/2026 – Versión 1.0

Resumen:

Zombie Survival es un videojuego de supervivencia por oleadas desarrollado en Python con la biblioteca Pygame, cuyo objetivo es integrar y demostrar los mecanismos de comunicación y sincronización entre hilos vistos en la asignatura. El jugador enfrenta rondas infinitas de zombies de dificultad creciente, con cuatro tipos de enemigos, un jefe con inteligencia artificial propia, obstáculos y potenciadores. La concurrencia se aplica en cuatro frentes: un hilo productor que genera las apariciones de enemigos comunicándose con el hilo principal mediante una cola sincronizada (patrón productor-consumidor), un pool de hilos que calcula en paralelo el movimiento de todos los zombies bajo un esquema fork-join con separación de fases de lectura y escritura, un hilo dedicado a la máquina de estados del jefe que comparte su vector de movimiento bajo exclusión mutua con *Lock*, y la protección de la vida del jugador ante modificaciones concurrentes. El resultado es un juego completo y funcional, listo para su utilización.

Palabras Clave: *concurrencia, hilos, sincronización, productor-consumidor, pygame*

I. CONTEXTO

El presente trabajo corresponde al Trabajo Práctico Integrador de la asignatura Programación Concurrente, del Departamento de Ingeniería e Investigaciones Tecnológicas de la Universidad Nacional de La Matanza (UNLaM), primer cuatrimestre del año 2026. El área de conocimiento es la programación concurrente y paralela, aplicada al desarrollo de videojuegos en tiempo real. La consigna requiere implementar una solución de software completa y funcional que integre procesos o hilos que se comuniquen y sincronicen entre sí [1].

II. INTRODUCCIÓN

La solución implementada es un videojuego 2D de supervivencia por oleadas (género *survival*) de un solo jugador. El jugador se desplaza por un mapa con obstáculos y debe eliminar zombies que lo persiguen, sobreviviendo la mayor cantidad de rondas posible. La dificultad escala indefinidamente: cada ronda trae más enemigos, con mayor frecuencia de aparición y mayor proporción de tipos difíciles. Cada cinco rondas aparece un jefe con comportamiento propio. El juego no tiene fin: la partida termina únicamente cuando la vida del jugador llega a cero.

La finalidad del proyecto es doble: por un lado, construir un juego completo y jugable; por el otro, ejercitar de forma concreta los mecanismos de la asignatura — creación y ciclo de vida de hilos, comunicación mediante colas sincronizadas, exclusión mutua con *locks*, señalización con eventos y paralelización de cómputo con pools de hilos.

Tecnologías utilizadas: lenguaje Python (versión 3.9 o superior, desarrollado y probado con 3.12), biblioteca Pygame 2.6.1 [2] para gráficos, ventana y eventos de entrada, y los módulos estándar `threading` [3], `queue` [4] y `concurrent.futures` [5] para la concurrencia. No utiliza dependencias adicionales ni servicios externos.

Restricciones: no posee restricciones de hardware (corre en cualquier equipo de escritorio moderno) ni de navegador (es una aplicación de escritorio). Es multiplataforma: funciona en Windows, Linux y macOS con solo tener Python instalado. La resolución lógica es de 900×600 píxeles, escalable a pantalla completa.

III. MÉTODOS

El proyecto se organizó en módulos con responsabilidades separadas: `game/` (bucle principal y estados del juego), `entities/` (jugador, zombies, jefe, balas, armas, potenciadores, obstáculos), `managers/` (rondas y assets), `factories/` (creación de zombies y armas a partir de configuración), `spawners/` (aparición de enemigos) y `settings.py` (configuración centralizada de balance). Se aplicaron los patrones de diseño *Factory* (creación de entidades desde diccionarios de configuración), *State* (estados del juego y de la ronda) y *Producer-Consumer*.

El bucle principal corre a 60 FPS en el hilo principal. Sobre esa base, la concurrencia se aplica en cuatro mecanismos:

A. Hilo generador de enemigos (productor-consumidor)

Cada ronda lanza un `SpawnerThread` (subclase de `threading.Thread`) que produce pedidos de aparición de zombies a intervalos regulares, independientemente del *frame rate*. El hilo elige el tipo de zombie por sorteo ponderado y lo deposita en una `queue.Queue`, estructura sincronizada de la biblioteca estándar que garantiza operaciones atómicas sin necesidad de locks explícitos. El hilo principal consume la cola una vez por cuadro con `get_nowait()` (lectura no bloqueante) y construye allí el objeto zombie, dado que las operaciones gráficas de Pygame no son seguras fuera del hilo principal: el productor decide *cuándo* y *qué* aparece; el consumidor lo *materializa*. La detención limpia se resuelve con `threading.Event`: la espera entre apariciones se hace con `event.wait(timeout)`, que duerme el intervalo configurado pero despierta inmediatamente si otro hilo señala el evento, permitiendo cancelar la ronda sin demoras.

B. Cálculo paralelo de la inteligencia artificial (fork-join)

El movimiento de cada zombie surge de la suma de cuatro fuerzas: atracción hacia el jugador, separación respecto de otros zombies, repulsión de obstáculos y una componente aleatoria. Este cómputo, de costo $O(n^2)$ por la separación entre pares, se paraleliza con un `ThreadPoolExecutor` de cuatro *workers*. La corrección se garantiza por separación de fases y no por copias ni locks: en la **fase paralela** cada worker ejecuta `compute_movement()`, una función de solo lectura que devuelve el vector de movimiento sin modificar estado; el hilo principal espera en barrera (`future.result()`) a que todos terminen; y recién en la **fase secuencial** aplica los movimientos y resuelve los ataques, uno por uno, en el hilo principal. Al no existir escrituras durante la fase de lectura, no hay condiciones de carrera posibles.

C. Hilo dedicado del jefe (exclusión mutua con Lock)

El jefe posee un hilo propio (`BossThread`) que ejecuta su máquina de estados a 20 Hz, desacoplada del bucle de juego: persecución (3,5 s) → advertencia (0,7 s, el jefe se detiene como señal visual) → embestida (0,9 s a 4,5× de velocidad en dirección fija). Este hilo **escribe** el par (dirección, multiplicador de velocidad) y los workers del pool lo **leen**; ambos accesos se protegen con un `threading.Lock` compartido. El lock no protege un valor aislado sino el **invariante** entre ambos: sin él, un lector podría observar la dirección nueva combinada con el multiplicador viejo, produciendo un estado inconsistente.

D. Protección de la vida del jugador

El método `take_damage()` del jugador realiza una secuencia *check-then-act*: consulta si hay un escudo activo, lo consume (absorbiendo el golpe) o descuenta vida. Toda la secuencia se ejecuta bajo un `threading.Lock` para garantizar su atomicidad

ante ataques concurrentes, protegiendo el diseño frente a futuras extensiones donde el daño pudiera aplicarse desde más de un hilo.

E. Ciclo de vida y limpieza de los hilos

Todos los hilos se crean como *daemon* y cada ronda expone un método `stop()` idempotente que señala el evento del productor, apaga el pool (`shutdown`) y detiene el hilo del jefe. Se invoca al completar la ronda, al morir el jugador, al reiniciar la partida y al cerrar el juego, evitando fugas de hilos entre rondas.

IV. RESULTADOS Y OBJETIVOS

Se obtuvo un juego completo, estable y listo para su utilización, que integra los cuatro mecanismos de concurrencia descritos funcionando en simultáneo durante toda la partida. Las funcionalidades logradas incluyen: rondas infinitas con escalado dinámico de dificultad; cuatro tipos de zombies (básico, rápido, corredor y tanque) con estadísticas y sprites diferenciados; jefe con máquina de estados propia cada cinco rondas, con indicador visual de fase y barra de vida; sistema de potenciadores con probabilidad de aparición al eliminar enemigos (cura, velocidad y escudo); obstáculos que los enemigos rodean, que bloquean balas y al jugador; HUD con vida, puntaje, ronda y efectos activos; pantallas de transición entre rondas y de fin de partida con reinicio; y modo pantalla completa.

Como objetivos futuros se identifican: implementar el sistema de armas múltiples ya previsto en la configuración (escopeta con múltiples proyectiles, dispersión y alcance), continuar el escalado de dificultad más allá de la ronda 13 (donde actualmente satura), optimizar la separación entre zombies con una grilla espacial para reducir el costo $O(n^2)$, y explorar `multiprocessing` para obtener paralelismo real del cómputo de IA, superando la limitación del GIL discutida en las conclusiones.

V. MANUAL DE USUARIO

A. Requisitos e instalación

Se requiere únicamente Python 3.9 o superior. Pasos:

```
# 1. Clonar el repositorio
git clone https://github.com/TeoFrancisTurri/survival-game.git
cd survival-game

# 2. Crear y activar un entorno virtual
python -m venv venv
venv\Scripts\activate # Windows
source venv/bin/activate # Linux / macOS

# 3. Instalar dependencias (pygame 2.6.1)
pip install -r requirements.txt

# 4. Ejecutar
python main.py # Windows
python3 main.py # Linux / macOS
```

B. Controles

TABLA I
CONTROLES DEL JUEGO

Tecla	Acción
W / A / S / D	Mover al jugador (arriba / izquierda / abajo / derecha)
ESPACIO	Disparar en la última dirección de movimiento (mantener presionado dispara en forma continua, limitado por el cooldown del arma)
F	Alternar pantalla completa
R	Reiniciar la partida (solo en la pantalla de Game Over)

C. Pantalla de juego

El HUD muestra en la esquina superior izquierda la **vida** (máximo 100), el **puntaje** acumulado y los **efectos activos** (línea "FX", en celeste); en la esquina superior derecha, la **ronda** actual y, cuando corresponde, el aviso rojo **RONDA DE JEFE**. Los bloques marrones son obstáculos: el jugador no puede atravesarlos, las balas se destruyen al impactarlos y los zombies intentan rodearlos.

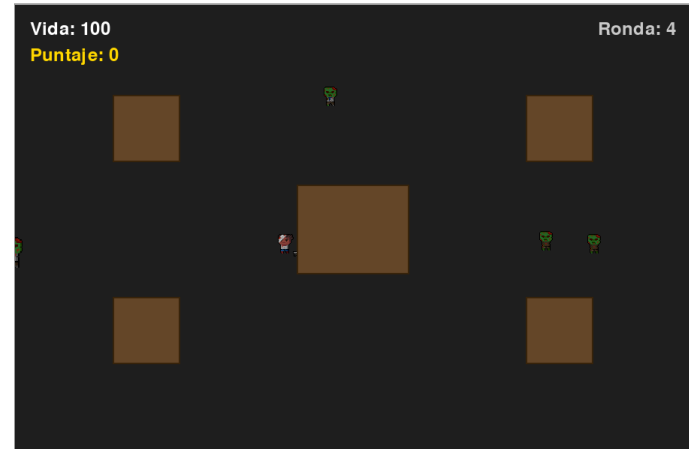


Fig. 1. Partida en curso. El jugador enfrenta zombies de distintos tipos que ingresan desde los bordes del mapa y rodean los obstáculos.

D. Mecánicas

Rondas: cada ronda tiene una cantidad fija de zombies que aparecen gradualmente desde los bordes. Al eliminarlos a todos, una pantalla de transición de 5 segundos anticipa la siguiente ronda (y avisa si la próxima es de jefe). La dificultad crece: más zombies, aparición más frecuente y tipos más peligrosos.

TABLA II
TIPOS DE ZOMBIE

Tipo	Tamaño	Velocidad	Vida	Daño	Puntos
Básico	Normal	Lenta	50	10	10
Rápido	Normal	Alta	30	8	15
Runner	Pequeño	Muy alta	15	6	20
Tank	Grande	Muy lenta	180	25	35

Potenciadores: al morir, cada zombie tiene un 25 % de probabilidad de dejar un ítem que desaparece a los 9 segundos.

Se recoge caminando sobre él: **CURA +25** (verde) restaura vida, **VEL ×1.5** (azul) acelera al jugador por 8 segundos y **ESCUDO** (amarillo) absorbe el próximo golpe recibido durante 10 segundos.

Rondas de jefe: en las rondas múltiplo de 5 aparece el Zombie Jefe (tamaño doble, 500 de vida, 200 puntos), que alterna tres fases indicadas por el color de su borde: persecución (rojo oscuro), advertencia (amarillo — se detiene antes de embestir) y embestida (rojo vivo — carga rápida en línea recta). La estrategia consiste en aprovechar la advertencia para esquivar la embestida.

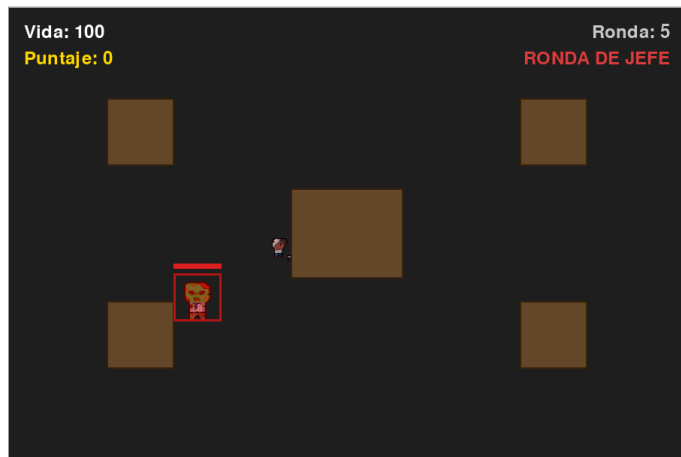


Fig. 2. Ronda de jefe. El Zombie Jefe con su barra de vida y el borde de color que indica la fase de su máquina de estados; el HUD anuncia "RONDA DE JEFE".

Fin de partida: cuando la vida llega a 0 se muestra la pantalla de Game Over con el puntaje final y la ronda alcanzada; la tecla R inicia una nueva partida.

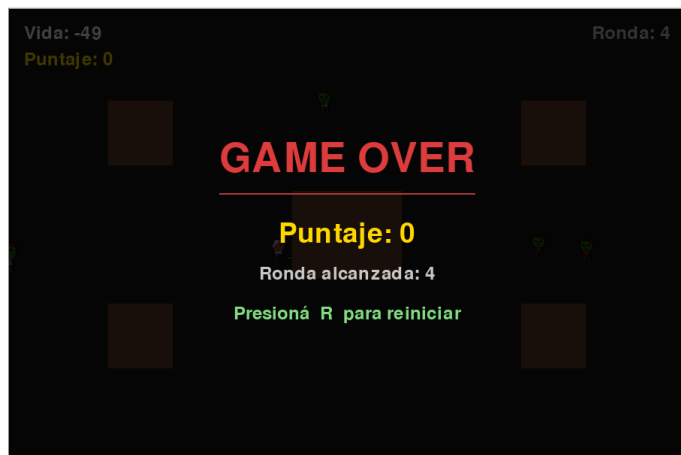


Fig. 3. Pantalla de Game Over con el puntaje final, la ronda alcanzada y la instrucción de reinicio.

VI. CONCLUSIONES

Las principales dificultades encontradas durante el desarrollo fueron las siguientes:

Pygame no es seguro entre hilos. Las primeras pruebas de crear zombies (con sus sprites) directamente desde el hilo productor generaban comportamientos erráticos, porque las

superficies gráficas de Pygame deben manipularse desde el hilo principal. La solución fue rediseñar el reparto de responsabilidades: el hilo productor solo decide *qué* y *cuándo* aparece y lo comunica por la cola; el hilo principal materializa el objeto. Esta dificultad terminó reforzando la aplicación correcta del patrón productor-consumidor.

Condiciones de carrera en el movimiento paralelo. Paralelizar la IA moviendo los zombies directamente desde los workers producía carreras de datos: cada zombie lee las posiciones de todos los demás para calcular su separación, mientras otros las modificaban. Detectarlo no fue trivial porque los efectos eran esporádicos (movimientos "temblosos" ocasionales). La solución fue separar el cálculo (paralelo, de solo lectura) de la aplicación (secuencial, en el hilo principal) con una barrera entre ambas fases — el esquema fork-join descrito en Métodos.

Consistencia del estado compartido del jefe. El vector de movimiento del jefe se compone de dos valores (dirección y multiplicador de velocidad) escritos por su hilo y leídos por el pool. Comprendimos que el lock debía abarcar el par completo como invariante y no cada valor por separado, para evitar lecturas de estados mezclados entre fases (por ejemplo, dirección de embestida con velocidad de persecución).

Detención limpia de los hilos. Al morir el jugador o reiniciar la partida quedaban hilos vivos de rondas anteriores. Se resolvió con `threading.Event` (las esperas con `wait(timeout)` despiertan de inmediato ante la señal de parada), hilos *daemon* y un método de detención idempotente invocado en todos los puntos de salida de la ronda.

La limitación del GIL. Aprendimos que en CPython el *Global Interpreter Lock* impide el paralelismo real de código Python puro: el pool de hilos aporta concurrencia y una arquitectura escalable, pero no acelera el cómputo de la IA, que es ligado a CPU. Los hilos sí resultan plenamente efectivos para esperas (como la del productor). Para paralelismo real, el camino sería `multiprocessing` o la vectorización del cálculo.

Como conclusión general, el proyecto permitió aplicar de punta a punta los contenidos de la asignatura sobre un problema real y observable: cada mecanismo de sincronización responde a una necesidad concreta del juego y su ausencia produce fallas visibles, lo que hizo tangible la teoría.

VII. REFERENCIAS Y BIBLIOGRAFÍA

- [1] Cátedra de Programación Concurrente, "Trabajo Práctico Integrador," Universidad Nacional de La Matanza, 2026.
- [2] P. Community, "Pygame Documentation," 2024. [en línea]. Consultado: julio 2026. Disponible: <https://www.pygame.org/docs/>
- [3] P. S. Foundation, "threading — Thread-based parallelism," Python 3 Documentation. [en línea]. Consultado: julio 2026. Disponible: <https://docs.python.org/3/library/threading.html>

[4] P. S. Foundation, "queue — A synchronized queue class," Python 3 Documentation. [en línea]. Consultado: julio 2026. Disponible:
<https://docs.python.org/3/library/queue.html>

[5] P. S. Foundation, "concurrent.futures — Launching parallel tasks," Python 3 Documentation. [en línea]. Consultado: julio 2026. Disponible:
<https://docs.python.org/3/library/concurrent.futures>